

8h às 10h

Explore os materiais e prepare o notebook-base.

Autoestudo: uma trilha antes de modelar

1

Separação de dados: treino e teste

2

Reprodutibilidade e replicabilidade

3

Primeiros passos com Scikit-Learn

4

Regressão linear e métricas

5

KNN, classificação e overfitting

6

Introdução a séries temporais

Stand-up daily: antes de transformar

15 min

10:00 → 10:15

1

Status

Qual material foi explorado?

2

Experimento

Qual célula foi executada?

3

Impedimento

Dado, código ou conceito?

Traga a mensagem de erro e a célula que a produziu.

DIAGRAMA DO MÓDULO



Aprendizado Supervisionado I

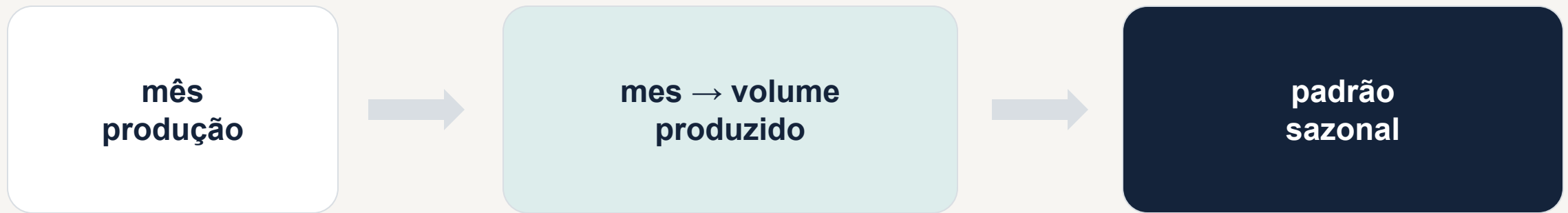
Hipóteses • Regressão • Classificação • Avaliação

Profª Dra. Crishna Irion

PARTE 1

Hipóteses

Uma hipótese conecta dados e problema



Hipótese: a produção varia entre os meses e pode revelar sazonalidade.

Quatro elementos tornam a hipótese verificável

1

Variável

o fator investigado
ex.: mês ou marca

2

Resultado

o que se explica
ex.: produção

3

Relação

o comportamento
esperado

4

Motivo

por que isso
faz sentido

Boa hipótese: específica, coerente com a base e acompanhada de evidência.

A hipótese não pode dizer mais do que a base mede

INVÁLIDA NESTA BASE

elétricos vendem mais

A planilha não registra vendas nem identifica a motorização efetiva.

INVESTIGÁVEL

Multi-energy cresce

É possível comparar a participação de plataformas ao longo dos meses.

Produção ≠ vendas. Plataforma Multi-energy ≠ classificação de veículo elétrico.

Três hipóteses podem orientar o projeto

1

Mês → produção

Meses diferentes
apresentam padrões
sazonais.

2

Marca → volume

Marcas possuem
capacidades produtivas
diferentes.

3

Histórico → previsão

Meses anteriores
ajudam a estimar
o mês seguinte.

Toda hipótese precisa de justificativa, análise, gráfico e interpretação.

PARTE 2

Fundamentos: modelos, dados e reprodutibilidade

Modelos Paramétricos e Não Paramétricos

Paramétricos

forma fixa, número de parâmetros não muda com os dados (ex.: Regressão Linear).

Não paramétricos

complexidade cresce com os dados, mais flexíveis (ex.: KNN, Árvores).

Trade-off: paramétrico tende a mais viés; não paramétrico tende a mais variância.

O tipo de resposta define a tarefa supervisionada

REGRESSÃO

prevê um número

quantos veículos?
qual será a produção?
qual será o faturamento?

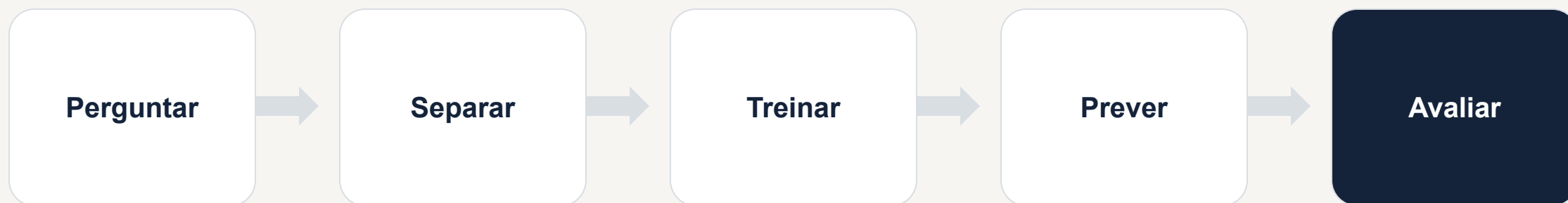
CLASSIFICAÇÃO

prevê uma categoria

renovou ou não?
sucesso ou fracasso?
alta ou baixa produção?

No projeto automotivo, prever quantidade mensal é um problema de regressão.

O primeiro ciclo supervisionado tem cinco decisões



```
modelo.fit(X_treino, y_treino)  
modelo.predict(X_teste)
```

Treino: dados usados para o modelo aprender os padrões dos dados.
Teste: dados usados para avaliar o modelo gerado.

Antes de treinar, identifique X e y

```
data = jun/2026  
mês = 6  
lag_1 = 293412  
lag_3 = média recente
```

O alvo é uma quantidade.

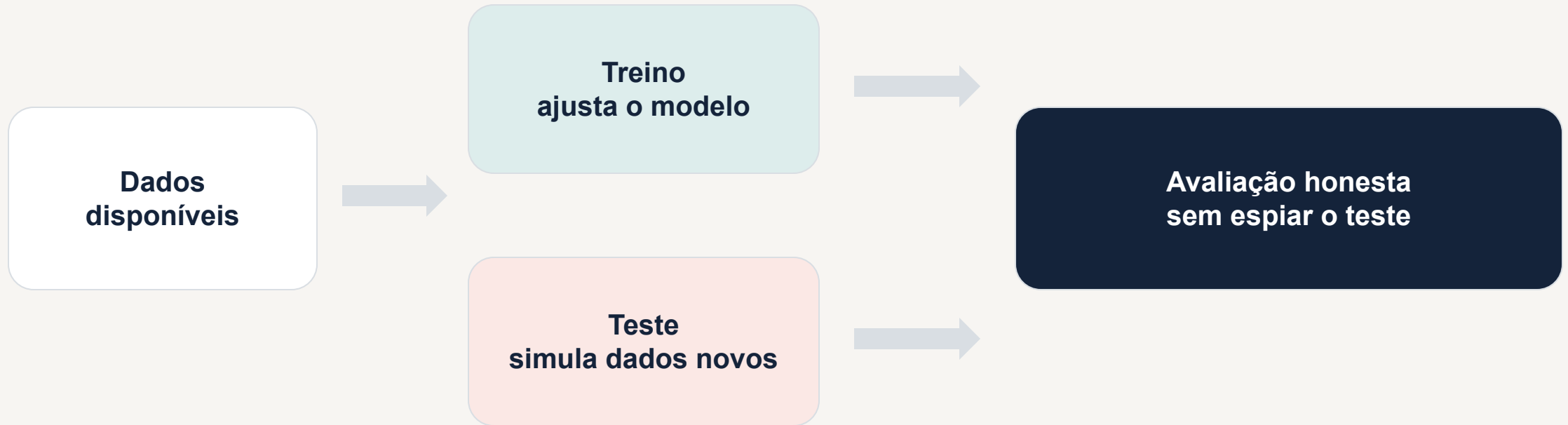
X = informações disponíveis
antes da previsão

y = produção do mês
que desejamos estimar

Cada linha representa um mês.

**Pergunta-chave: esse dado existia no momento da
decisão?**

Treino aprende; teste verifica generalização



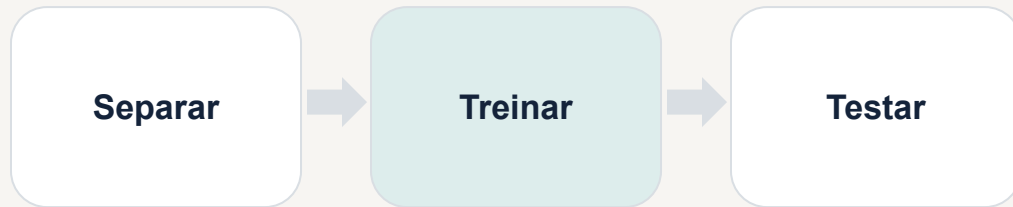
O teste não participa do ajuste, da imputação nem da escolha das regras.

PARTE 3

Na prática: classificação, regressão e séries temporais

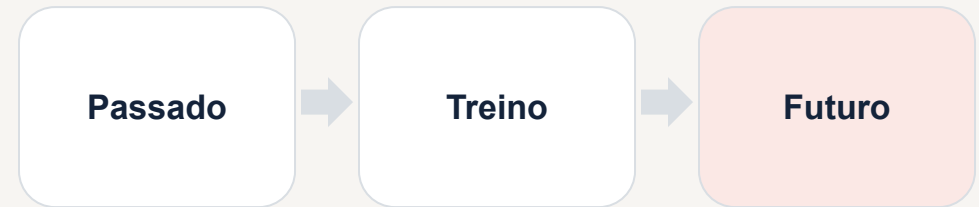
Séries temporais: o passado vem antes do futuro

DADOS SEM ORDEM



Divisão aleatória pode ser aceitável.

SÉRIE TEMPORAL



O teste deve acontecer depois do treino.

Embaralhar uma série pode misturar o futuro no treinamento e inflar a avaliação.

O que Muda em Relação aos Outros Modelos

- A ordem temporal importa: nunca embaralhe os dados antes de dividir.
- Componentes: tendência, sazonalidade, ciclos e ruído.
- Treino = passado, teste = futuro (respeitar a cronologia).
- Use `TimeSeriesSplit` para validação cruzada respeitando o tempo.

```
TimeSeriesSplit(n_splits=4)
```

Resultados confiáveis precisam ser reproduzíveis

seed = 42

**Mesmos dados + mesmo código +
mesmas condições.**

- 1 Organizar**
ordem clara e células executáveis
- 2 Controlar**
sementes, versões e parâmetros
- 3 Registrar**
dados, escolhas, métricas e limites

Regressão linear

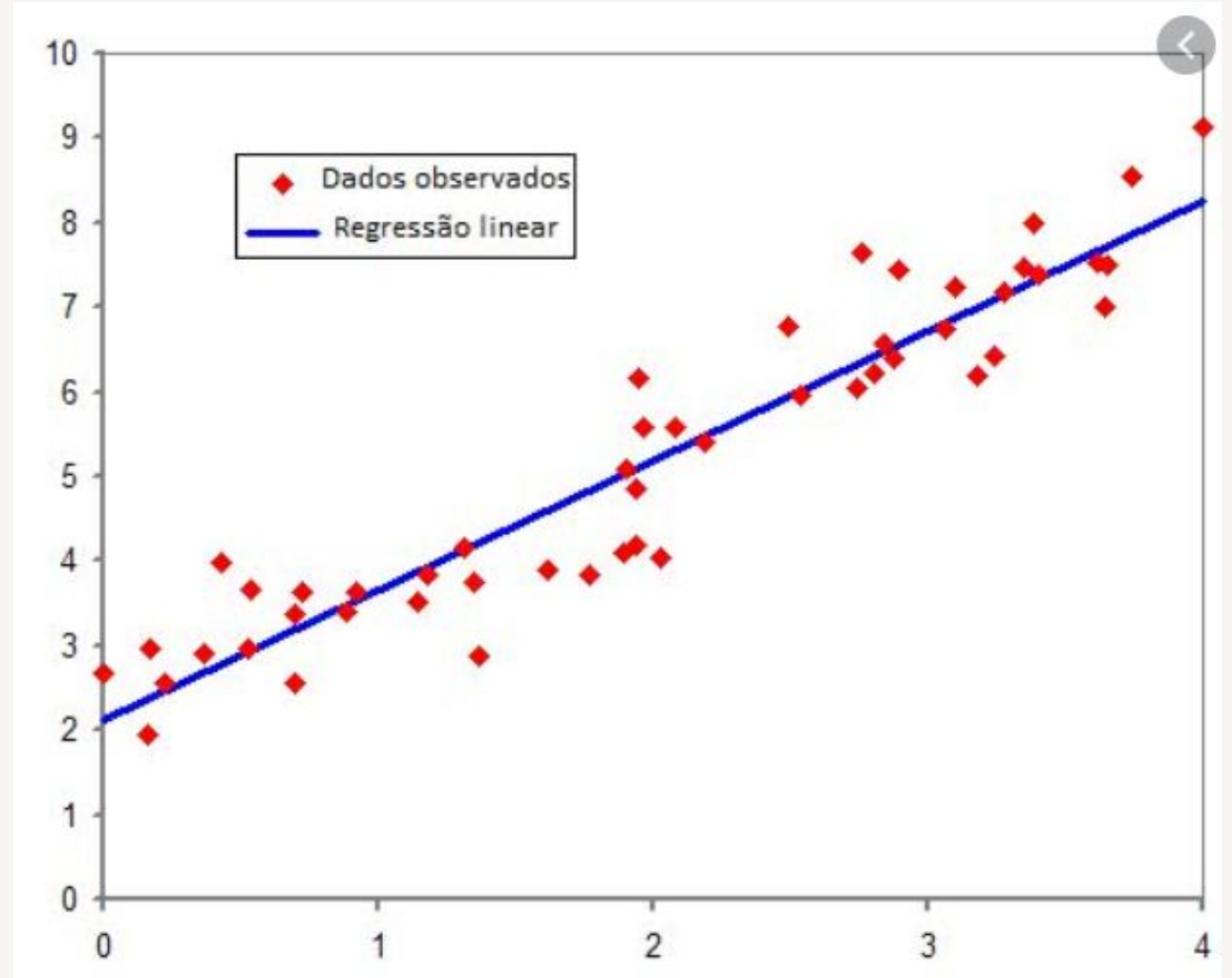
UMA VARIÁVEL

$$\hat{y} = b_0 + b_1x$$

x : característica conhecida

$\hat{y}$: quantidade estimada

b_0 e b_1 : parâmetros ajustados



Com scikit-learn, treinar e prever tem um padrão

```
modelo = LinearRegression()  
  
modelo.fit(X_treino, y_treino)  
  
y_previsto = modelo.predict(X_teste)
```

fit aprende
predict estima
métrica compara

A entrada do teste precisa seguir as mesmas transformações aplicadas ao treino.

Modelo ajustado não é modelo aprovado: ainda falta avaliar.

Todo modelo precisa superar uma referência simples

baseline ingênuo

$$\hat{y}_t = y_{t-1}$$

modelo linear

$$\hat{y}_t = f(X_t)$$

Comparação justa

Avalie baseline e modelo nos mesmos meses e com a mesma métrica. Se a referência simples vence, o modelo ainda não agregou valor.

Boa previsão não é apenas rodar código: é melhorar uma alternativa plausível.

Avaliação de Modelos — Regressão

1

MAE

erro absoluto médio
na mesma unidade
do problema

2

MSE

erro quadrático médio
dá peso maior
a erros grandes

3

R²

compara o ajuste
com uma referência
baseada na média

MAE e MSE: quanto menor, melhor. R²: interpretar junto com o contexto.

Generalizar é equilibrar simplicidade e ajuste

1

Underfitting

modelo simples demais
erra no treino
e também no teste

2

Bom ajuste

aprende padrões
sem depender
de coincidências

3

Overfitting

vai bem no treino
mas piora
em dados novos

Compare treino e teste; não escolha o modelo apenas pelo erro de treinamento.

Classificação com KNN - Usa os vizinhos mais próximos

k = 3

o modelo vai olhar para os três pontos de dados mais próximos (os vizinhos mais pertos) de um novo item para decidir a qual grupo ele pertence

1

Medir

distância entre os exemplos

2

Selecionar

os k vizinhos mais próximos

3

Votar

escolher a classe predominante

`KNeighborsClassifier(n_neighbors=k)`

Exemplo Prático: KNN

k = 3

Pipeline completo:

1. Separar treino/teste com stratify e random_state fixo.
2. Normalizar as features com StandardScaler.
3. Escolher o melhor K via validação cruzada (cross_val_score).
4. Avaliar: acurácia, matriz de confusão, classification_report.

```
KNeighborsClassifier(n_neighbors=k)
```

Métricas de Classificação

ACURÁCIA

Porcentagem de acertos totais do modelo

Precisão (Precision)

Quantas previsões positivas foram corretas de fato.

Recall

Quanto casos positivos reais o modelo conseguiu achar.

F1-Score

Média harmônica entre precisão e recall.

MATRIZ DE CONFUSÃO

Tabela que mostra acertos e erros por classe e ajuda a entender falsos positivos e negativos

Curva ROC e AUC

Gráfico que mede a capacidade de separar classes.

Parâmetros e Hiperparâmetros

Parâmetros

- Pesos (weights) e vieses (biases).
- Ajustados automaticamente pelos dados.
- Definem o conhecimento final do modelo treinado.

Hiperparâmetros

- Taxa de aprendizado (learning rate).
- Número de épocas ou camadas ocultas.
- Definidos manualmente pelo cientista de dados.

PARTE 5

Mão na Massa

Bora?



Kahoot!

Desafio



<https://encurtador.com.br/hgDo>

Hipótese testada. Previsão defendida.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

